Chapter 4

Intel 8086

8086 Microprocessor is an enhanced version of 8085Microprocessor that was designed by Intel in 1976. It is a 16-bit Microprocessor having 20 address lines and16 data lines that provides up to 1MB storage. It consists of powerful instruction set, which provides operations like multiplication and division easily.

It supports two modes of operation, i.e. Maximum mode and Minimum mode. Maximum mode is suitable for system having multiple processors and Minimum mode is suitable for system having a single processor.

Features of 8086

The most prominent features of an 8086 microprocessor are as follows −

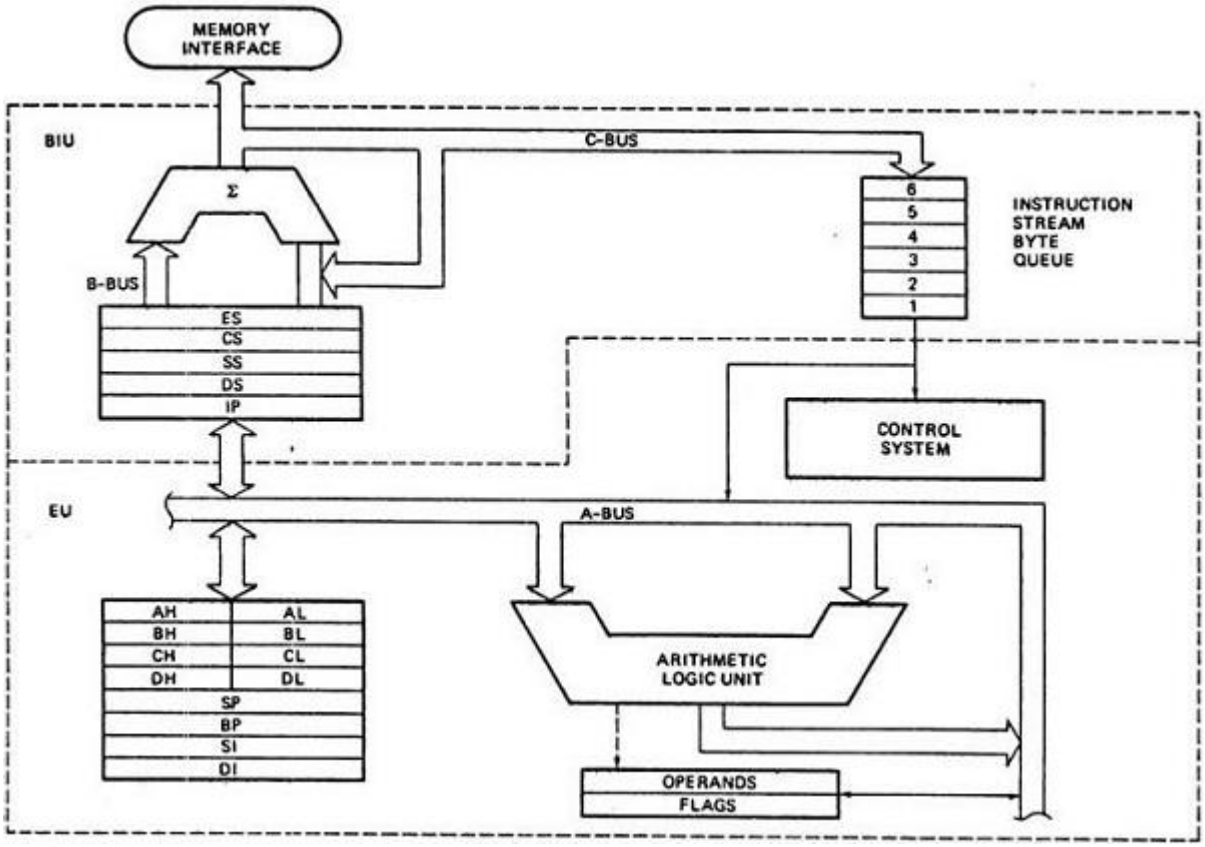
- It has an instruction queue, which is capable of storing six instruction bytes from the memory resulting in faster processing.
- It was the first 16-bit processor having 16-bit ALU, 16-bit registers, internal data bus, and 16-bit external data bus resulting in faster processing.
- It is available in 3 versions based on the frequency of operation −
    - 8086 → 5MHz
    - 8086-2 → 8MHz
    - (c)8086-1 → 10 MHz
- It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage, which improves performance.
- Fetch stage can prefetch up to 6 bytes of instructions and stores them in the queue.
- Execute stage executes these instructions.
- It has 256 vectored interrupts.
- It consists of 29,000 transistors.

Comparison between 8085 & 8086 Microprocessor

- **Size** − 8085 is 8-bit microprocessor, whereas 8086 is 16-bit microprocessor.
- **Address Bus** − 8085 has 16-bit address bus while 8086 has 20-bit address bus.
- **Memory** − 8085 can access up to 64Kb, whereas 8086 can access up to 1 Mb of memory.
- **Instruction** − 8085 doesn’t have an instruction queue, whereas 8086 has an instruction queue.
- **Pipelining** − 8085 doesn’t support a pipelined architecture while 8086 supports a pipelined architecture.

- **I/O** – 8085 can address 2^8 = 256 I/O's, whereas 8086 can access 2^16 = 65,536 I/O's.
- **Cost** – The cost of 8085 is low whereas that of 8086 is high.

Architecture of 8086



8086 Microprocessor is divided into two functional units, i.e., **EU** (Execution Unit) and **BIU** (Bus Interface Unit).

EU (Execution Unit)

Execution unit gives instructions to BIU stating from where to fetch the data and then decode and execute those instructions. Its function is to control operations on data using the instruction decoder & ALU. EU has no direct connection with system buses as shown in the above figure, it performs operations over data through BIU.

Let us now discuss the functional parts of 8086 microprocessors.

ALU

It handles all arithmetic and logical operations, like +, −, ×, /, OR, AND, NOT operations.

Flag Register

It is a 16-bit register that behaves like a flip-flop, i.e. it changes its status according to the result stored in the accumulator. It has 9 flags and they are divided into 2 groups − Conditional Flags and Control Flags.

Conditional Flags

It represents the result of the last arithmetic or logical instruction executed. Following is the list of conditional flags :

- **Carry flag** − This flag indicates an overflow condition for arithmetic operations.
- **Auxiliary flag** − When an operation is performed at ALU, it results in a carry/barrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), then this flag is set, i.e. carry given by D3 bit to D4 is AF flag. The processor uses this flag to perform binary to BCD conversion.
- **Parity flag** − This flag is used to indicate the parity of the result, i.e. when the lower order 8-bits of the result contains even number of 1's, then the Parity Flag is set. For odd number of 1's, the Parity Flag is reset.
- **Zero flag** − This flag is set to 1 when the result of arithmetic or logical operation is zero else it is set to 0.
- **Sign flag** − This flag holds the sign of the result, i.e. when the result of the operation is negative, then the sign flag is set to 1 else set to 0.
- **Overflow flag** − This flag represents the result when the system capacity is exceeded.

Control Flags

Control flags controls the operations of the execution unit. Following is the list of control flags −

- **Trap flag** − It is used for single step control and allows the user to execute one instruction at a time for debugging. If it is set, then the program can be run in a single step mode.
- **Interrupt flag** − It is an interrupt enable/disable flag, i.e. used to allow/prohibit the interruption of a program. It is set to 1 for interrupt enabled condition and set to 0 for interrupt disabled condition.
- **Direction flag** − It is used in string operation. As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-a-versa.

General purpose register

There are 8 general purpose registers, i.e., AH, AL, BH, BL, CH, CL, DH, and DL. These registers can be used individually to store 8-bit data and can be used in pairs to store 16bit data. The valid register pairs are AH and AL, BH and BL, CH and CL, and DH and DL. It is referred to the AX, BX, CX, and DX respectively.

- **AX register** − It is also known as accumulator register. It is used to store operands for arithmetic operations.
- **BX register** − It is used as a base register. It is used to store the starting base address of the memory area within the data segment.
- **CX register** − It is referred to as counter. It is used in loop instruction to store the loop counter.
- **DX register** − This register is used to hold I/O port address for I/O instruction.

Stack pointer register

It is a 16-bit register, which holds the address from the start of the segment to the memory location, where a word was most recently stored on the stack.

BIU (Bus Interface Unit)

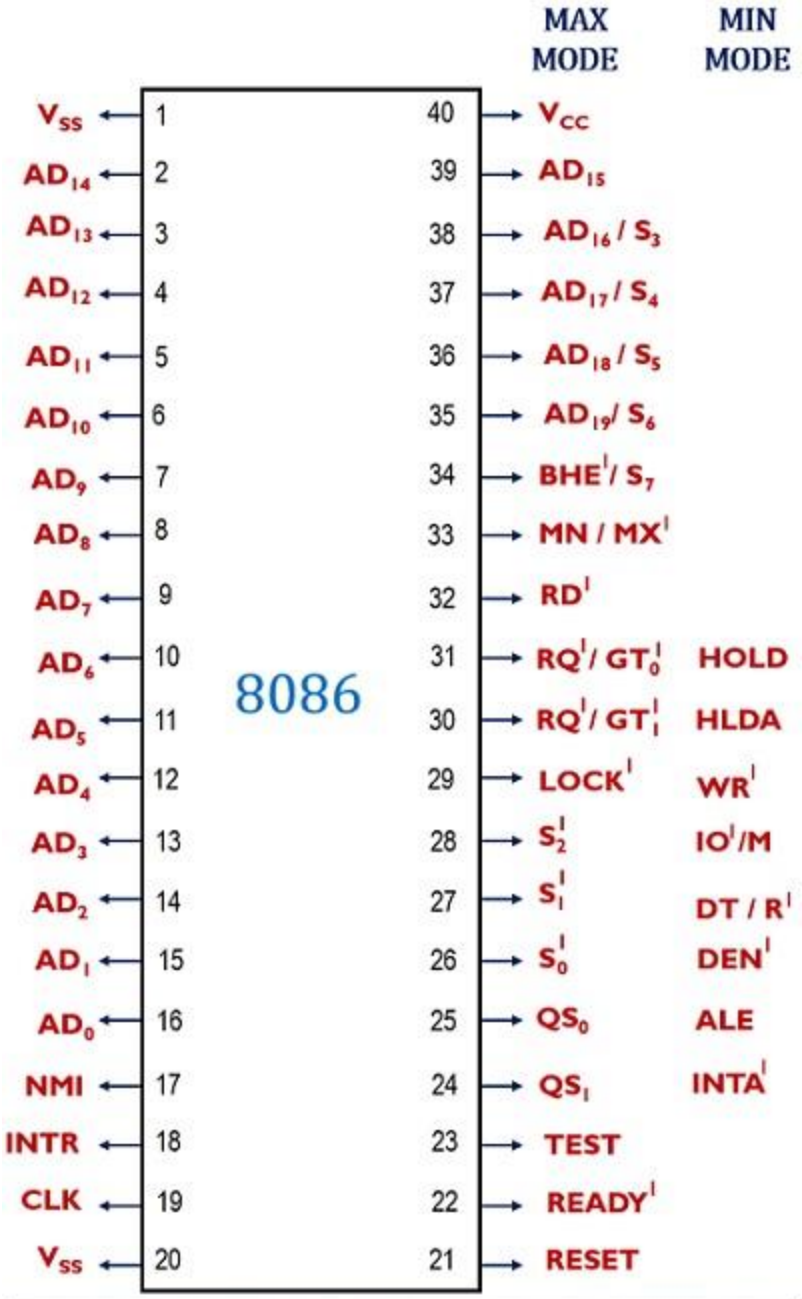
BIU takes care of all data and addresses transfers on the buses for the EU like sending addresses, fetching instructions from the memory, reading data from the ports and the memory as well as writing data to the ports and the memory. EU has no direction connection with System Buses so this is possible with the BIU. EU and BIU are connected with the Internal Bus.

It has the following functional parts −

- **Instruction queue** − BIU contains the instruction queue. BIU gets up to 6 bytes of next instructions and stores them in the instruction queue. When EU executes instructions and is ready for its next instruction, then it simply reads the instruction from this instruction queue resulting in increased execution speed.
- Fetching the next instruction while the current instruction executes is called **pipelining**.
- **Segment register** − BIU has 4 segment buses, i.e. CS, DS, SS& ES. It holds the addresses of instructions and data in memory, which are used by the processor to access memory locations. It also contains 1 pointer register IP, which holds the address of the next instruction to execute by the EU.
    - **CS** − It stands for Code Segment. It is used for addressing a memory location in the code segment of the memory, where the executable program is stored.
    - **DS** − It stands for Data Segment. It consists of data used by the program andis accessed in the data segment by an offset address or the content of other register that holds the offset address.
    - **SS** − It stands for Stack Segment. It handles memory to store data and addresses during execution.
    - **ES** − It stands for Extra Segment. ES is additional data segment, which is used by the string to hold the extra destination data.
- **Instruction pointer** − It is a 16-bit register used to hold the address of the next instruction to be executed.

8086 was the first 16-bit microprocessor available in 40-pin DIP (Dual Inline Package) chip. Let us now discuss in detail the pin configuration of an 8086 Microprocessor.

8086 Pin Diagram



Pin diagram of 8086 Microprocessor

Power supply and frequency signals

It uses 5V DC supply at V_{CC} pin 40, and uses ground at V_{SS} pin 1 and 20 for its operation.

Clock signal

Clock signal is provided through Pin-19. It provides timing to the processor for operations. Its frequency is different for different versions, i.e. 5MHz, 8MHz and 10MHz.

Address/data bus

AD0-AD15. These are 16 address/data bus. AD0-AD7 carries low order byte data and AD8AD15 carries higher order byte data. During the first clock cycle, it carries 16-bit address and after that it carries 16-bit data.

Address/status bus

A16-A19/S3-S6. These are the 4 address/status buses. During the first clock cycle, it carries 4-bit address and later it carries status signals.

S7/BHE

BHE stands for Bus High Enable. It is available at pin 34 and used to indicate the transfer of data using data bus D8-D15. This signal is low during the first clock cycle, thereafter it is active.

Read$(\overline{\mathbf{RD}})$

It is available at pin 32 and is used to read signal for Read operation.

Ready

It is available at pin 22. It is an acknowledgement signal from I/O devices that data is transferred. It is an active high signal. When it is high, it indicates that the device is ready to transfer data. When it is low, it indicates wait state.

RESET

It is available at pin 21 and is used to restart the execution. It causes the processor to immediately terminate its present activity. This signal is active high for the first 4 clock cycles to RESET the microprocessor.

INTR

It is available at pin 18. It is an interrupt request signal, which is sampled during the last clock cycle of each instruction to determine if the processor considered this as an interrupt or not.

NMI

It stands for non-maskable interrupt and is available at pin 17. It is an edge triggered input, which causes an interrupt request to the microprocessor.

Test$(\overline{\mathbf{TEST}})$

This signal is like wait state and is available at pin 23. When this signal is high, then the processor has to wait for IDLE state, else the execution continues.

$\mathbf{MN/\overline{MX}}$

It stands for Minimum/Maximum and is available at pin 33. It indicates what mode the processor is to operate in; when it is high, it works in the minimum mode and vice-versa.

$\mathbf{\overline{INTA}}$

It is an interrupt acknowledgement signal and id available at pin 24. When the microprocessor receives this signal, it acknowledges the interrupt.

ALE

It stands for address enable latch and is available at pin 25. A positive pulse is generated each time the processor begins any operation. This signal indicates the availability of a valid address on the address/data lines.

$\mathbf{\overline{DEN}}$

It stands for Data Enable and is available at pin 26. It is used to enable Transreceiver 8286. The transreceiver is a device used to separate data from the address/data bus.

$\mathbf{DT/\overline{R}}$

It stands for Data Transmit/Receive signal and is available at pin 27. It decides the direction of data flow through the transreceiver. When it is high, data is transmitted out and vice-a-versa.

$\mathbf{M/\overline{IO}}$

This signal is used to distinguish between memory and I/O operations. When it is high, it indicates I/O operation and when it is low indicates the memory operation. It is available at pin 28.

$\mathbf{\overline{WR}}$

It stands for write signal and is available at pin 29. It is used to write the data into the memory or the output device depending on the status of M/IO signal.

HLDA

It stands for Hold Acknowledgement signal and is available at pin 30. This signal acknowledges the HOLD signal.

HOLD

This signal indicates to the processor that external devices are requesting to access the address/data buses. It is available at pin 31.

QS_1 and QS_0

These are queue status signals and are available at pin 24 and 25. These signals provide the status of instruction queue. Their conditions are shown in the following table −

QS_0	QS_1	Status
0	0	No operation
0	1	First byte of opcode from the queue
1	0	Empty the queue
1	1	Subsequent byte from the queue

$\overline{S_0}, \overline{S_1}, \overline{S_2}$

These are the status signals that provide the status of operation, which is used by the Bus Controller 8288 to generate memory & I/O control signals. These are available at pin 26, 27, and 28. Following is the table showing their status −

S_2	S_1	S_0	Status
0	0	0	Interrupt acknowledgement
0	0	1	I/O Read
0	1	0	I/O Write
0	1	1	Halt
1	0	0	Opcode fetch
1	0	1	Memory read
1	1	0	Memory write
1	1	1	Passive

$\overline{\mathbf{LOCK}}$

When this signal is active, it indicates to the other processors not to ask the CPU to leave the system bus. It is activated using the LOCK prefix on any instruction and is available at pin 29.

$\overline{\boldsymbol{RQ}}/\overline{\boldsymbol{GT1}}$ ***and*** $\overline{\boldsymbol{RQ}}/\overline{\boldsymbol{GT0}}$

These are the Request/Grant signals used by the other processors requesting the CPU to release the system bus. When the signal is received by CPU, then it sends acknowledgment. $\overline{RQ}/\overline{GT0}$ has a higher priority than $\overline{RQ}/\overline{GT1}$.

Instructions Set

The 8086 microprocessor supports 8 types of instructions

- Data Transfer Instructions
- Arithmetic Instructions
- Bit Manipulation Instructions
- String Instructions
- Program Execution Transfer Instructions (Branch & Loop Instructions)
- Processor Control Instructions
- Iteration Control Instructions
- Interrupt Instructions

Let us now discuss these instruction sets in detail.

Data Transfer Instructions

These instructions are used to transfer the data from the source operand to the destination operand. Following are the list of instructions under this group −

Instruction to transfer a word

- **MOV** − Used to copy the byte or word from the provided source to the provided destination.
- **PPUSH** − Used to put a word at the top of the stack.
- **POP** − Used to get a word from the top of the stack to the provided location.
- **PUSHA** − Used to put all the registers into the stack.
- **POPA** − Used to get words from the stack to all registers.
- **XCHG** − Used to exchange the data from two locations.
- **XLAT** − Used to translate a byte in AL using a table in the memory.

Instructions for input and output port transfer

- **IN** − Used to read a byte or word from the provided port to the accumulator.
- **OUT** − Used to send out a byte or word from the accumulator to the provided port.

Instructions to transfer the address

- **LEA** − Used to load the address of operand into the provided register.
- **LDS** − Used to load DS register and other provided register from the memory
- **LES** − Used to load ES register and other provided register from the memory.

Instructions to transfer flag registers

- **LAHF** − Used to load AH with the low byte of the flag register.
- **SAHF** − Used to store AH register to low byte of the flag register.
- **PUSHF** − Used to copy the flag register at the top of the stack.
- **POPF** − Used to copy a word at the top of the stack to the flag register.

Arithmetic Instructions

These instructions are used to perform arithmetic operations like addition, subtraction, multiplication, division, etc.

Following is the list of instructions under this group −

Instructions to perform addition

- **ADD** − Used to add the provided byte to byte/word to word.
- **ADC** − Used to add with carry.
- **INC** − Used to increment the provided byte/word by 1.
- **AAA** − Used to adjust ASCII after addition.
- **DAA** − Used to adjust the decimal after the addition/subtraction operation.

Instructions to perform subtraction

- **SUB** − Used to subtract the byte from byte/word from word.
- **SBB** − Used to perform subtraction with borrow.
- **DEC** − Used to decrement the provided byte/word by 1.
- **NPG** − Used to negate each bit of the provided byte/word and add 1/2's complement.
- **CMP** − Used to compare 2 provided byte/word.
- **AAS** − Used to adjust ASCII codes after subtraction.
- **DAS** − Used to adjust decimal after subtraction.

Instruction to perform multiplication

- **MUL** − Used to multiply unsigned byte by byte/word by word.
- **IMUL** − Used to multiply signed byte by byte/word by word.
- **AAM** − Used to adjust ASCII codes after multiplication.

Instructions to perform division

- **DIV** − Used to divide the unsigned word by byte or unsigned double word by word.
- **IDIV** − Used to divide the signed word by byte or signed double word by word.
- **AAD** − Used to adjust ASCII codes after division.
- **CBW** − Used to fill the upper byte of the word with the copies of sign bit of the lower byte.
- **CWD** − Used to fill the upper word of the double word with the sign bit of the lower word.

Bit Manipulation Instructions

These instructions are used to perform operations where data bits are involved, i.e. operations like logical, shift, etc.

Following is the list of instructions under this group −

Instructions to perform logical operation

- **NOT** − Used to invert each bit of a byte or word.
- **AND** − Used for adding each bit in a byte/word with the corresponding bit in another byte/word.
- **OR** − Used to multiply each bit in a byte/word with the corresponding bit in another byte/word.
- **XOR** − Used to perform Exclusive-OR operation over each bit in a byte/word with the corresponding bit in another byte/word.
- **TEST** − Used to add operands to update flags, without affecting operands.

Instructions to perform shift operations

- **SHL/SAL** − Used to shift bits of a byte/word towards left and put zero(S) in LSBs.
- **SHR** − Used to shift bits of a byte/word towards the right and put zero(S) in MSBs.
- **SAR** − Used to shift bits of a byte/word towards the right and copy the old MSB into the new MSB.

Instructions to perform rotate operations

- **ROL** − Used to rotate bits of byte/word towards the left, i.e. MSB to LSB and to Carry Flag [CF].
- **ROR** − Used to rotate bits of byte/word towards the right, i.e. LSB to MSB and to Carry Flag [CF].
- **RCR** − Used to rotate bits of byte/word towards the right, i.e. LSB to CF and CF to MSB.
- **RCL** − Used to rotate bits of byte/word towards the left, i.e. MSB to CF and CF to LSB.

String Instructions

String is a group of bytes/words and their memory is always allocated in a sequential order.

Following is the list of instructions under this group −

- **REP** − Used to repeat the given instruction till CX ≠ 0.
- **REPE/REPZ** − Used to repeat the given instruction until CX = 0 or zero flag ZF = 1.

- **REPNE/REPNZ** − Used to repeat the given instruction until CX = 0 or zero flag ZF = 1.
- **MOVS/MOVSB/MOVSW** − Used to move the byte/word from one string to another.
- **COMS/COMPSB/COMPSW** − Used to compare two string bytes/words.
- **INS/INSB/INSW** − Used as an input string/byte/word from the I/O port to the provided memory location.
- **OUTS/OUTSB/OUTSW** − Used as an output string/byte/word from the provided memory location to the I/O port.
- **SCAS/SCASB/SCASW** − Used to scan a string and compare its byte with a byte in AL or string word with a word in AX.
- **LODS/LODSB/LODSW** − Used to store the string byte into AL or string word into AX.

Program Execution Transfer Instructions (Branch and Loop Instructions)

These instructions are used to transfer/branch the instructions during an execution. It includes the following instructions −

Instructions to transfer the instruction during an execution without any condition −

- **CALL** − Used to call a procedure and save their return address to the stack.
- **RET** − Used to return from the procedure to the main program.
- **JMP** − Used to jump to the provided address to proceed to the next instruction.

Instructions to transfer the instruction during an execution with some conditions

- **JA/JNBE** − Used to jump if above/not below/equal instruction satisfies.
- **JAE/JNB** − Used to jump if above/not below instruction satisfies.
- **JBE/JNA** − Used to jump if below/equal/ not above instruction satisfies.
- **JC** − Used to jump if carry flag CF = 1
- **JE/JZ** − Used to jump if equal/zero flag ZF = 1
- **JG/JNLE** − Used to jump if greater/not less than/equal instruction satisfies.
- **JGE/JNL** − Used to jump if greater than/equal/not less than instruction satisfies.
- **JL/JNGE** − Used to jump if less than/not greater than/equal instruction satisfies.
- **JLE/JNG** − Used to jump if less than/equal/if not greater than instruction satisfies.
- **JNC** − Used to jump if no carry flag (CF = 0)
- **JNE/JNZ** − Used to jump if not equal/zero flag ZF = 0
- **JNO** − Used to jump if no overflow flag OF = 0
- **JNP/JPO** − Used to jump if not parity/parity odd PF = 0
- **JNS** − Used to jump if not sign SF = 0
- **JO** − Used to jump if overflow flag OF = 1

- **JP/JPE** − Used to jump if parity/parity even PF = 1
- **JS** − Used to jump if sign flag SF = 1

Processor Control Instructions

These instructions are used to control the processor action by setting/resetting the flag values.

Following are the instructions under this group −

- **STC** − Used to set carry flag CF to 1
- **CLC** − Used to clear/reset carry flag CF to 0
- **CMC** − Used to put complement at the state of carry flag CF.
- **STD** − Used to set the direction flag DF to 1
- **CLD** − Used to clear/reset the direction flag DF to 0
- **STI** − Used to set the interrupt enable flag to 1, i.e., enable INTR input.
- **CLI** − Used to clear the interrupt enable flag to 0, i.e., disable INTR input.

Iteration Control Instructions

These instructions are used to execute the given instructions for number of times. Following is the list of instructions under this group −

- **LOOP** − Used to loop a group of instructions until the condition satisfies, i.e., CX = 0
- **LOOPE/LOOPZ** − Used to loop a group of instructions till it satisfies ZF = 1 & CX = 0
- **LOOPNE/LOOPNZ** − Used to loop a group of instructions till it satisfies ZF = 0 & CX = 0
- **JCXZ** − Used to jump to the provided address if CX = 0

Interrupt Instructions

These instructions are used to call the interrupt during program execution.

- **INT** − Used to interrupt the program during execution and calling service specified.
- **INTO** − Used to interrupt the program during execution if OF = 1
- **IRET** − Used to return from interrupt service to the main program

Addressing modes of 8086

Source: https://www.ic.unicamp.br/~celio/mc404s2-03/addr_modes/intel_addr.html#:~:text=There%20are%20a%20total%20of,disp%5Bbp%5D%5Bdi%5D.

1. **Register Addressing Mode**

   Most 8086 instructions can operate on the 8086's general purpose register set. By specifying the name of the register as an operand to the instruction, you may access the contents of that register. Consider the 8086 MOV (move) instruction:

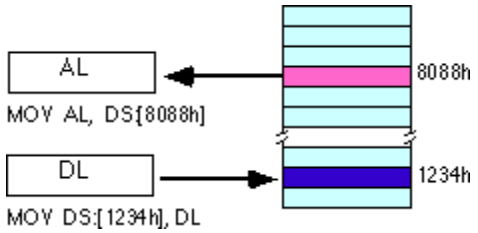
   MOV destination, source

   This instruction copies the data from the source operand to the destination operand. The eight and 16 bit registers are certainly valid operands for this instruction. The only restriction is that both operands must be the same size. There are two restrictions on the use of the segment registers with the MOV instruction. First of all, you may not specify CS as the destination operand, second, only one of the operands can be a segment register. You cannot move data from one segment register to another with a single MOV instruction. To copy the value of CS to CS, you'd have to use some sequence like:

   ```
   MOV    AX, CS
   MOV    DS, AX
   ```

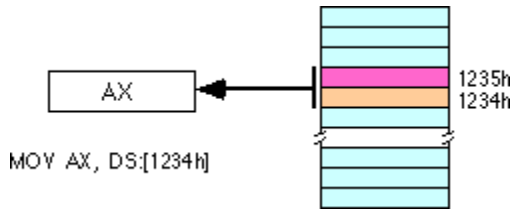
2. **The displacement only addressing mode**

   The displacement-only addressing mode consists of a 16 bit constant that specifies the address of the target location. The instruction MOV AL,DS:[8088H] loads the AL register with a copy of the byte at memory location 8088h. Likewise, the instruction MOV DS:[1234H],DL stores the value in the DL register to memory location 1234h:



   The displacement-only addressing mode is perfect for accessing simple variables. Of course, you'd probably prefer using names like "I" or "J" rather than "DS:[1234h]" or "DS:[8088h]". Well, fear not, you'll soon see it's possible to do just that.

   Intel named this the displacement-only addressing mode because a 16 bit constant (displacement) follows the MOV opcode in memory. In that respect it is quite similar to the direct addressing mode on the x86 processors. There are some minor differences, however. First of all, a displacement is exactly that- some distance from some other point. On the x86, a direct address can be thought of as a displacement from address zero. On the 80x86 processors, this displacement is an offset from the beginning of a segment (the data segment in this example). You can think of the displacement-only addressing mode as a direct addressing mode. The examples in this chapter will typically access bytes in memory. Don't forget, however, that you can also access words on the 8086 processors:



By default, all displacement-only values provide offsets into the data segment. If you want to provide an offset into a different segment, you must use a segment override prefix before your address. For example, to access location 1234h in the extra segment (ES) you would use an instruction of the form MOV AX,ES:[1234H]. Likewise, to access this location in the code segment you would use the instruction MOV AX, CS:[1234H]. The DS: prefix in the previous examples is not a segment override. The CPU uses the data segment register by default.

3. The Register Indirect Addressing Modes

The 80x86 CPUs let you access memory indirectly through a register using the register indirect addressing modes. There are four forms of this addressing mode on the 8086, best demonstrated by the following instructions:

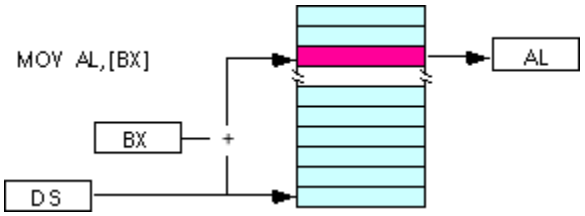
```
MOV    AL, [BX]
MOV    AL, [BP]
MOV    AL, [SI]
MOV    AL, [DI]
```

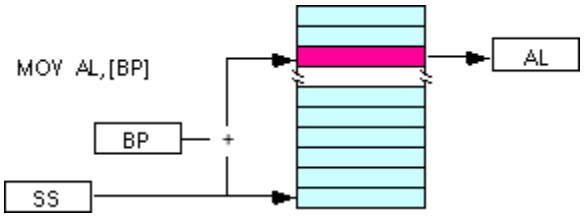
As with the x86 [BX] addressing mode, these four addressing modes reference the byte at the offset found in the BX, BP, SI, or DI register, respectively. The [BX], [SI], and [DI] modes use the DS segment by default. The [BP] addressing mode uses the stack segment (SS) by default.

You can use the segment override prefix symbols if you wish to access data in different segments. The following instructions demonstrate the use of these overrides:

```
MOV    AL, CS:[BX]
MOV    AL, DS:[BP]
MOV    AL, SS:[SI]
MOV    AL, ES:[DI]
```

Intel refers to [BX] and [BP] as base addressing modes and BX and BP as base registers (in fact, BP stands for base pointer). Intel refers to the [SI] and [DI] addressing modes as indexed addressing modes (SI stands for source index, DI stands for destination index). However, these addressing modes are functionally equivalent. This text will call these forms register indirect modes to be consistent.





Note: the [SI] and [DI] addressing modes work exactly the same way, just substitute SI and DI for BX above.

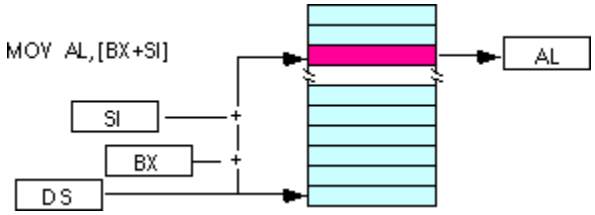
4. Base Indexed Addressing Modes

The based indexed addressing modes are simply combinations of the register indirect addressing modes. These addressing modes form the offset by adding together a base register (BX or BP) and an index register (SI or DI). The allowable forms for these addressing modes are

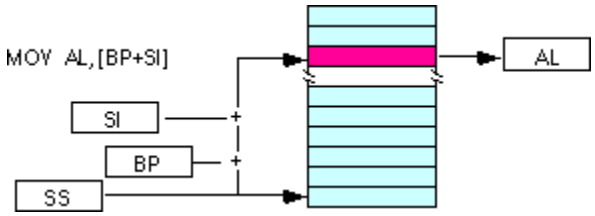
```
MOV    AL, [BX][SI]
MOV    AL, [BX][DI]
MOV    AL, [BP][SI]
MOV    AL, [BP][DI]
```

Suppose that BX contains 1000H and SI contains 880H. Then the instruction MOV AL,[BX][SI] would load AL from location DS:1880H. Likewise, if BP contains 1598H and DI contains 1004, MOV AX,[BP+DI] will load the 16 bits in AX from locations SS:259C and SS:259D.

The addressing modes that do not involve BP use the data segment by default. Those that have BP as an operand use the stack segment by default.



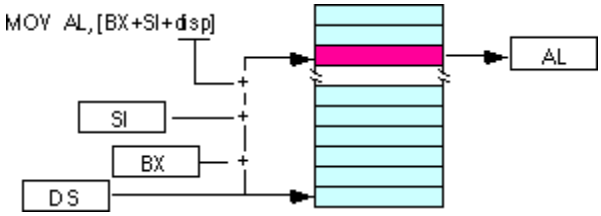
You substitute DI in the figure above to obtain the [BX+DI] addressing mode.



You substitute DI in the figure above for the [BP+DI] addressing mode.

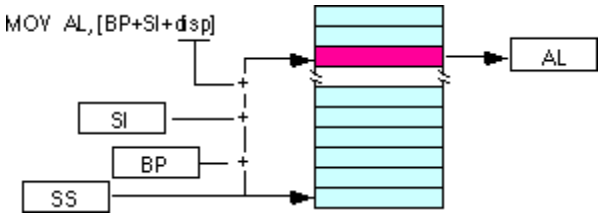
5. Based Indexed Plus Displacement Addressing Mode

These addressing modes are a slight modification of the base/indexed addressing modes with the addition of an eight bit or sixteen bit constant. The following are some examples of these addressing modes:




```
MOV     AL, DISP[BX][SI]
MOV     AL, DISP[BX+DI]
MOV     AL, [BP+SI+DISP]
MOV     AL, [BP][DI][DISP]
```

You may substitute DI in the figure above to produce the [BX+DI+DISP] addressing mode.



You may substitute DI in the figure above to produce the [BP+DI+DISP] addressing mode.

Suppose BP contains 1000h, BX contains 2000H, SI contains 120H, and DI contains 5. Then MOV AL,10H[BX+SI] loads AL from address DS:2130; MOV CH,125H[BP+DI] loads CH from location SS:112A; and MOV BX,CS:2[BX][DI] loads BX from location CS:2007.

An Easy Way to Remember the 8086 Memory Addressing Modes

There are a total of 17 different legal memory addressing modes on the 8086: DISP, [BX], [BP], [SI], [DI], DISP[BX], DISP[BP], DISP[SI], DISP[DI], [BX][SI], [BX][DI], [BP][SI], [BP][DI], DISP[BX][SI], DISP [BX][DI], DISP[BP][SI], AND DISP[BP][DI]. You could memorize all these forms so that you know which are valid (and, by omission, which forms are invalid). However, there is an easier way besides memorizing these 17 forms. Consider the chart:

DISP	[BX]	[SI]
	[BP]	[DI]

If you choose zero or one items from each of the columns and wind up with at least one item, you've got a valid 8086 memory addressing mode. Some examples:

- Choose DISP from column one, nothing from column two, [DI] from column 3, you get DISP[DI].
- Choose DISP, [BX], and [DI]. You get DISP[BX][DI].
- Skip column one & two, choose [SI]. You get [SI]
- Skip column one, choose [BX], then choose [DI]. You get [BX][DI]

Likewise, if you have an addressing mode that you cannot construct from this table, then it is not legal. For example, DISP[DX][SI] is illegal because you cannot obtain [DX] from any of the columns above.

Difference between Macro and Procedure

MACRO	PROCEDURE
Sequence of instructions that is written within the macro definition to support modular programming	Set of instructions which can be called repetitively that performs a specific task
Requires more memory	Requires less memory
Does not require CALL and RET instructions	Requires CALL and RET instructions
Machine code is generated each time the macro is called	Machine code generates only once
Parameters are passed as a part of statement which calls the macro	Parameters are passed in registers and memory locations of stack
Macro executes faster than a procedure	Procedure executes slower than a macro
Eliminates the overhead time to call the procedure and to return the program	Requires more overhead time to call the procedure and to return back to the calling procedure
Used for less than 10 instructions	Used for more than 10 instructions

Microsoft Macro Assembler (MASM)

The Microsoft Macro Assembler (MASM) is an x86 assembler that uses the Intel syntax for MS-DOS and Microsoft windows.

DOS INT 21H- DOS Function codes

AH	Description	AH	Description
01	Read character from STDIN	02	Write character to STDOUT
05	Write character to printer	06	Console Input/Output
07	Direct char read (STDIN), no echo	08	Char read from STDIN, no echo
09	Write string to STDOUT	0A	Buffered input
0B	Get STDIN status	0C	Flush buffer for STDIN
0D	Disk reset	0E	Select default drive
19	Get current default drive	25	Set interrupt vector
2A	Get system date	2B	Set system date
2C	Get system time	2D	Set system time
2E	Set verify flag	30	Get DOS version
35	Get Interrupt vector		
36	Get free disk space	39	Create subdirectory
3A	Remove subdirectory	3B	Set working directory
3C	Create file	3D	Open file
3E	Close file	3F	Read file
40	Write file	41	Delete file
42	Seek file	43	Get/Set file attributes
47	Get current directory	4C	Exit program
4D	Get return code	54	Get verify flag
56	Rename file	57	Get/Set file date

Source: http://spike.scu.edu.au/~barry/interrupts.html

AH = 01h - READ CHARACTER FROM STANDARD INPUT, WITH ECHO

Return: AL = character read

AH = 02h -WRITE CHARACTER TO STANDARD OUTPUT

Entry: DL = character to write

Return: AL = last character output

AH = 09h - WRITE STRING TO STANDARD OUTPUT

Entry: DS:DX -> '$'-terminated string

Return: AL = 24h

AH = 4Ch - "EXIT" - TERMINATE WITH RETURN CODE

Entry: AL = return code

Return: never returns

Difference between one pass and two pass assembler:

One Pass Assembler	Two Pass Assembler
Scans entire source file only once	Require two passes to scan source file. **First pass** – responsible for label definition and introduce them in symbol table. **Second pass** – translates the instructions into assembly language or generates machine code.
Generally • Deals with syntax. • Constructs symbol table • Creates label list. • Indentifies the code segment, data segment, stack segment etc…	Along with pass1 pass two is also required which • Generates actual Opcode. • Compute actual address of every label. • Assign code address for debugging the information. • Translates operand name to appropriate register or memory code. • Immediate value is translated to binary strings (1's and 0's)
Cannot resolve forward references of data symbols.	Can resolve forward references of data symbols.
No object program is written, hence no loader is required	Loader is required as object code is generated.
Tends to be faster compared to two pass	Two pass assembler requires rescanning. Hence slow compared to one pass assembler.
Only creates tables with all symbols no address of symbols is calculated.	Address of symbols can be calculated

Assembler Directive

An assembler directive is a statement to give direction to the assembler to perform task of the assembly process. It controls the organization of the program and provide necessary information to the assembler to understand the assembly language programs to generate necessary machine codes. They indicate how an operand or a section of the program is to be processed by the assembler.

Few Assembler Directives of the 8086 Microprocessor

- The DB directive
- The DW directive
- The DD directive
- The EQU Directive
- DUP
- ASSUME
- SEGMENT
- OFFSET
- PROC

Data declaration directives:

1. DB – The DB directive is used to declare a BYTE -2-BYTE variable – A BYTE is made up of 8 bits.

   Declaration examples:

   Byte1 DB 10h

   Byte2 DB 255

2. DW – The DW directive is used to declare a WORD type variable – A WORD occupies 16 bits or (2 BYTE).

   Declaration examples:

   Word DW 1234h

   Word2 DW 65535; 0FFFFh, (the max. possible for a WORD)

3. DD – The DD directive is used to declare a DWORD – A DWORD double word is made up of 32 bits =2 Words or 4 BYTE.

   Declaration examples:

   Dword1 DW 12345678h

   Dword2 DW 4294967295 ;0FFFFFFFFh.

4. STRUCT and ENDS directives to define a structure template for grouping data items.

   The STRUCT directive tells the assembler that a user defined uninitialized data structure follows. The uninitialized data structure consists of a combination of the three supported data types. DB, DW, and DD. The labels serve as zero-based offsets into the structure. The first element's offset for any structure is 0. A structure element is referenced with the base "+" operator before the element's name.

   A Structure ends by using the ENDS directive meaning END of Structure.

   Syntax:

STRUCT

Structure_element_name element_data_type?

. . .

. . .

. . .

ENDS

(OR)

STRUC

Structure_element_name element_data_type?

. . .

. . .

. . .

ENDS

DECLARATION:

STRUCT

Byte1 DB?

Byte2 DB?

Word1 DW?

Word2 DW?

Dword1DW?

Dword2 DW?

ENDS

Use OF STRUCT:

The STRUCT directive enables us to change the order of items in the structure when, we reform a file header and shuffle the data. Shuffle the data items in the file header and reformat the sequence of data declaration in the STRUCT and off you go. No change in the code we write that processes the file header is necessary unless you inserted an extra data element.

5. The EQU Directive

The EQU directive is used to give name to some value or symbol. Each time the assembler finds the given names in the program, it will replace the name with the value or a symbol. The value can be in the range 0 through 65535 and it can be another Equate declared anywhere above or below.

The following operators can also be used to declare an Equate:

THIS BYTE

THIS WORD

THIS DWORD

A variable – declared with a DB, DW, or DD directive – has an address and has space reserved at that address for it in the .COM file. But an Equate does not have an address or space reserved for it in the .COM file.

Example:

A – Byte EQU THIS BYTE

DB 10

A_ word EQU THIS WORD

DW 1000

A_ dword EQU THIS DWORD

DD 4294967295

Buffer Size EQU 1024

Buffer DB 1024 DUP (0)

6. PROC: (PROCEDURE)

   It is used to identify the start of a procedure. It follows a name we give the procedure.

   After the procedure the term NEAR and FAR is used to specify the procedure Example: SMART-DIVIDE PROC FAR identifies the start of procedure named SMART-DIVIDE and tells the assembler that the procedure is far.

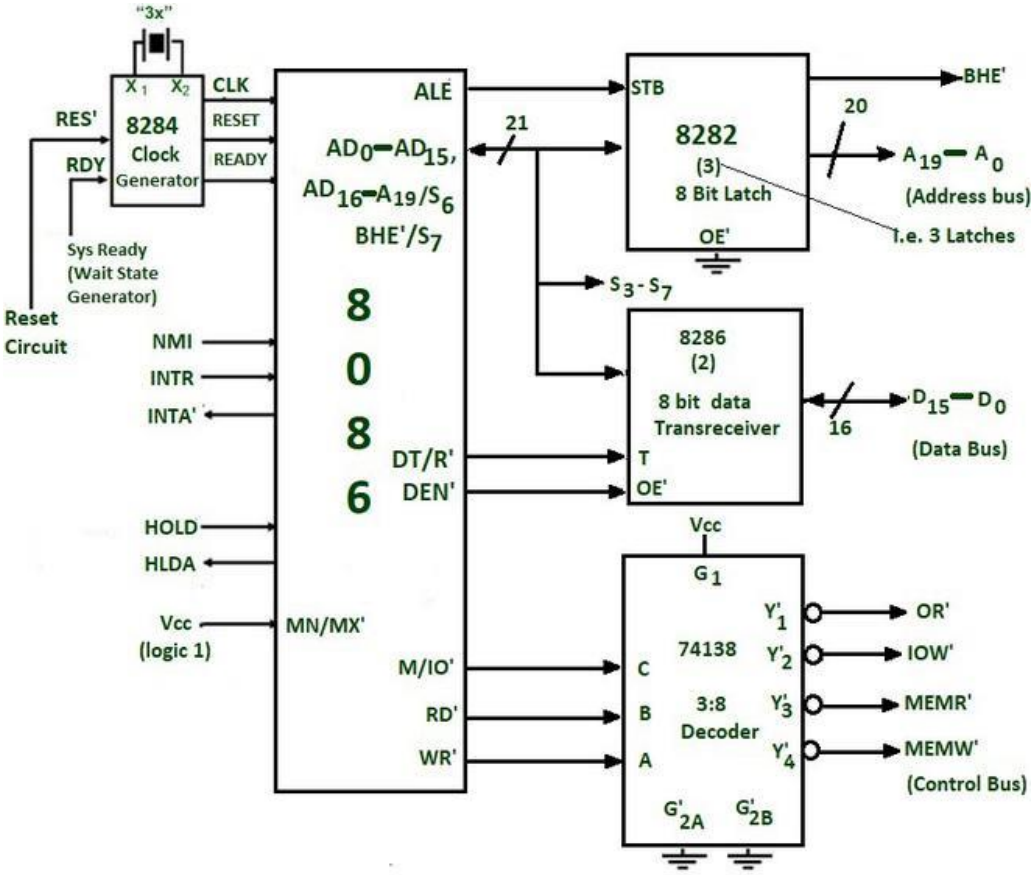
7. OFFSET:

   It is an operator which tells the assembler to determine the offset or displacement of a named data item from the start of the segment which contains it. It is used to load the offset of a variable into a register so that variable can be accessed with one of the addressed modes. Example: when the assembler read MOV BX, OFFSET PRICES, it will determine the offset of the prices.

8. DUP
   It is an assembler directive which informs assembler to duplicate data. Programmer must provide how many times duplication should be made.
   Variable_name DB 100 DUP (0) ; This will duplicate 0 in 100 locations.

System Timing Diagram

Control signals provided by 8086 for memory operations and I/O interfacing:
They are used to identifying whether the bus is carrying a valid address or not , in which direction data is needed to be transferred over the bus, when there is valid write data on the data bus and when to put read data on the system bus. Therefore, their sequence pattern makes all the operations successful in a particular machine cycle.



Min mode circuit

8282 (8 bits) latch:
The latches are buffered D FF. They are used to separate the valid address from the multiplexed Address/data bus by using the control signal ALE, which is connected to strobe (STB) of 8282. The ALE is active high signal. Here three such latches are required because the address is 20 bits.

8286 (8 bits) transceivers:
They are bidirectional buffers and also known as data amplifiers. They are used to separate the valid data from multiplexed add/data bus. Two such transceivers are needed because the data bus is 16 bits long. 8286 is connected to DT/R' and DEN' signals. They are enabled through the DEN signal .The direction of data on the data bus is controlled by the DT/R' signal. DT/R' is connected to T and DEN' is connected to OE'.

DEN'	DT/R'	Action
1	X (don't care)	Transreceiver is disabled
0	0	Receiver data
0	1	Transmit data

- 8284 clock generator is used to provide the clock.
- M/IO'= 1, then M transfer is performed over the bus and when M/IO' = 0, then I/O operation is performed.
- The signals RD' and write WR' are used to identify whether a read bus cycle or a write bus cycle is performing. When WR' = 0, then it indicates that valid output data on the data bus.
- RD' indicates that the 8086 is performing a read data or instruction fetch process is occurring. During read operations, one other control signal is also used, which is DEN (data enable) and it indicates the external devices when they should put data on the bus.
- Control signals for all operations are generated by decoding M/IO', RD', WR'. They are decoded by 74138 3:8 decoder.

M/IO'	RD'	WR'	Action
1	0	1	Memory Read
1	1	0	Memory Write
0	0	1	I/O Read
0	1	0	I/O Write

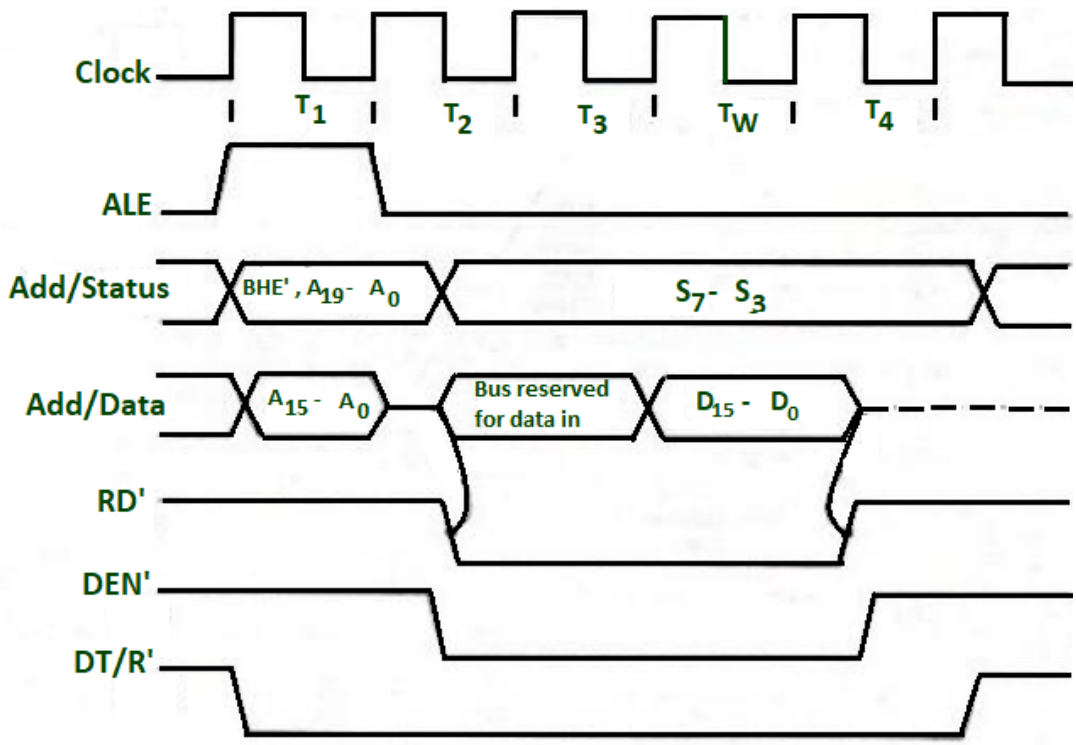
INTR and INTA:
When INTR = 1, then there is an interrupt to 8086 by other devices for their service. When INTA'= 0, then it indicates that the processor is ready to service them.

The bus request is made by other devices using the HOLD signal and the processor acknowledges them using the HLDA output signal.

Timing diagram:
The working of min mode can be easily understood by timing diagrams.

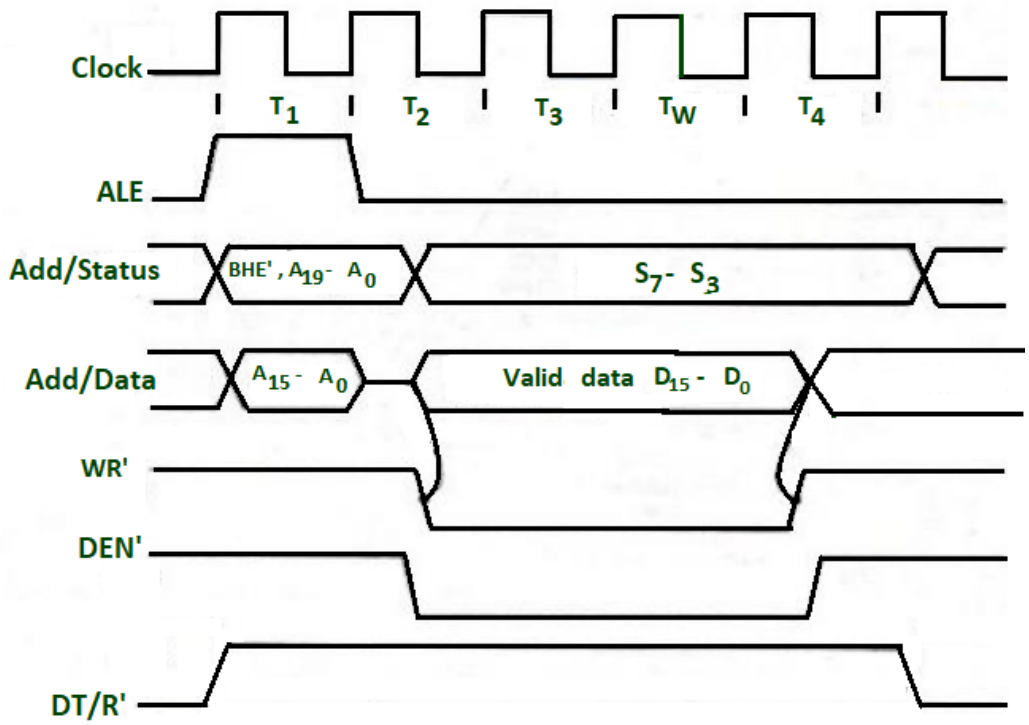
- All processors bus cycle is of at least 4 T-states (T1, T2, T3, T4) .The address is given by processor in the T1 state. It is available on the bus for one T-state.
- In T2, the bus is tristated for changing the direction of the bus (in the case of a data read cycle).
- The data transfer takes place between T3 and T4.
- If the addressed device is slower, then the wait state is inserted between T3 and T4.

Opcode fetch or read timing diagram



- At T1 state ALE =1, this indicates that a valid address is latched on the address bus and also M / IO'= 1, which indicates the memory operation is in progress.
- In T2, the address is removed from the local bus and is sent to the addressed device. Then the bus is tristated.
- When RD' = 0, the valid data is present on the data bus.
- During T2 DEN' =0, which enables transceivers and DT/R' = 0, which indicates that the data is received.
- During T3, data is put on the data bus and the processor reads it.
- The output device makes the READY line high. This means the output device has performed the data transfer process. When the processor makes the read signal to 1, then the output device will again tristate its bus drivers.

Write memory cycle



- At T1 state ALE =1, this indicates that a valid address is latched on the address bus and also M / IO'= 1, which indicates the memory operation is in progress.
- In T2, the processor sends the data to be written to the addressed location.
- The data is buffered on the bus until the middle of T4 state.
- The WR'=0 becomes at the beginning of T2.
- The BHE' and A0 signals are used to select the byte or bytes of memory or I/O word.
- During T2 DEN' =0, which enables, transceivers and DT/R' = 1, which indicates that the data is transferred by the processor to the addressed device.

Interrupt and Interrupt service procedure

An Interrupt Structure of 8086 can come from any one the three sources:

- External signal
- Special Instruction in the program
- Condition produced by instruction

1. External Signal (Hardware Interrupt):

   An 8086 can get interrupt from an external signal applied to the Non-Maskable Interrupt (NMI) input pin or the interrupt (INTR) input pin.

2. Special Instruction:

   Interrupt Structure of 8086 supports a special instruction, INT to execute special program. At the end of the interrupt service routine, execution is usually returned to the interrupted program.

3. Condition Produced by Instruction:

   An 8086 is interrupted by some condition produced in the 8086 by the execution of an instruction. For example divide by zero: Program execution will automatically be interrupted if you attempt to divide an operand by zero.

   At the end of each instruction cycle 8086 interrupts checks to see if there is any interrupt request. If so, 8086 responds to the interrupt by performing series of actions.

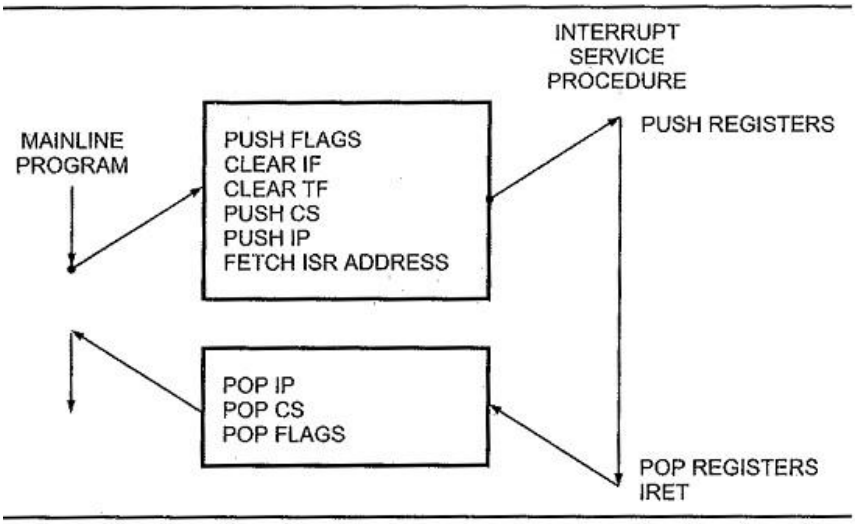


Fig. 9.1 8086 interrupt response

I. It decrements stack pointer by 2 and pushes the flag register on the stack.
II. It disables the INTR interrupt input by clearing the interrupt flag in the flag
III. It resets the trap flag in the flag register.
IV. It decrements stack pointer by 2 and pushes the current code segment register contents on the stack.
V. It decrements stack pointer by 2 and pushes the current instruction pointer contents on the stack.
VI. It does an indirect far jump at the start of the procedure by loading the CS and IP values for the start of the interrupt service routine (ISR).

Interrupt Vector Table (IVT)

Now the question is “How to get the values of CS and IP register?” The 8086 gets the new values of CS and IP register from four memory addresses. When it responds to an interrupt, the 8686 goes to memory locations to get the CS and IP values for the start of the interrupt service routine. In an Interrupt Structure of 8086 system the first 1 Kbyte of memory from 00000H to 003FFH is reserved for storing the starting addresses of interrupt service routines. This block of memory is often called the Interrupt Vector Table in 8086 or the interrupt pointer table. Since 4 bytes are required to store the CS and IP values for each interrupt service procedure, the table can hold the starting addresses for 256 interrupt service routines. Figure below shows how the 256 interrupt pointers are arranged in the memory table.

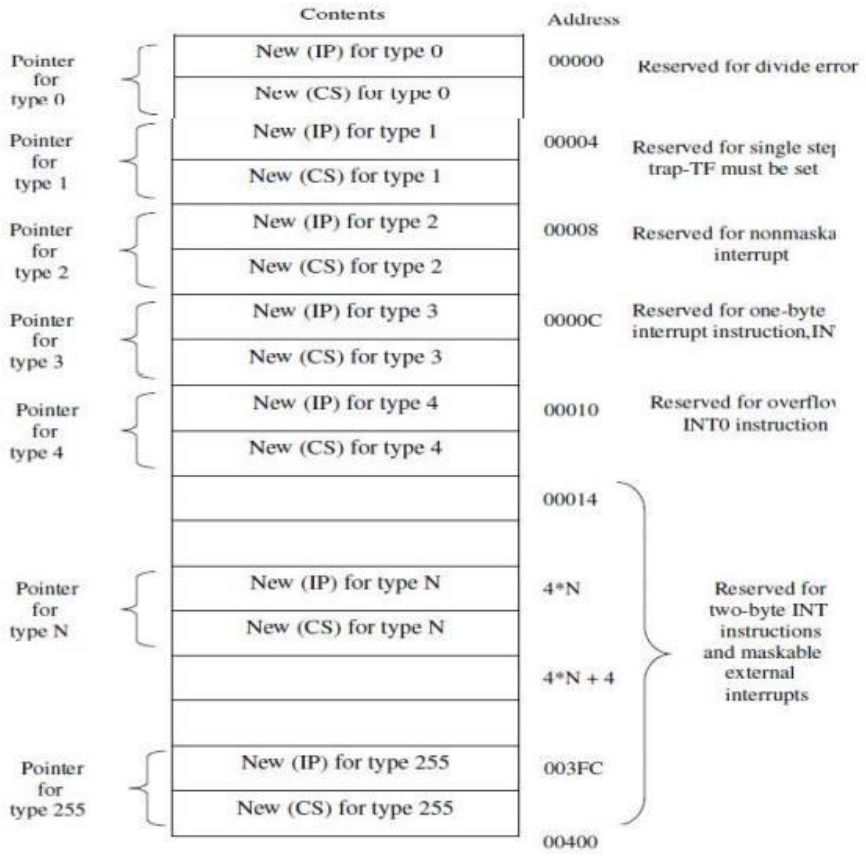


Fig. 1.12 Organization of Interrupt vector table in 8086

Introduction to 80386

Features of 80386

- As it is a 32-bit microprocessor. Thus has 32-bit ALU.
- 80386 has data bus of 32-bit.
- It holds address bus of 32 bit.
- It supports physical memory addressability of 4 GB and virtual memory addressability of 64 TB.

- 80386 supports variety of operating clock frequency, which are 16 MHz, 20 MHz, 25 MHz and 33 MHz.
- It offers 3 stage pipeline: fetch, decode and execute. As it supports simultaneous fetching, decoding and execution inside the system.